Sistema Gralha

Documentação Técnica de Software

Gerenciamento de Partituras e Arquivos Musicais

Versão 1.0

22 de março de 2026

Sumário

1 Introdução **3**
1.1 Objetivo do Documento . . . 3
1.2 Escopo do Sistema . . . 3
1.3 Definições e Termos . . . 3

2 Visão Geral do Sistema **4**
2.1 Descrição Geral . . . 4
2.2 Problema que o Sistema Resolve . . . 4
2.3 Público-Alvo . . . 4
2.4 Funcionalidades Principais . . . 5

3 Tecnologias Utilizadas **5**
3.1 Linguagens de Programação . . . 5
3.2 Bibliotecas e Frameworks . . . 5
3.3 Banco de Dados . . . 6
3.4 Ferramentas de Desenvolvimento . . . 6

4 Arquitetura do Sistema **7**
4.1 Visão Geral da Arquitetura . . . 7
4.2 Estrutura de Diretórios . . . 7
4.3 Organização dos Módulos . . . 7
4.4 Diagrama da Arquitetura . . . 8

5 Modelagem de Dados **8**
5.1 Estrutura do Banco de Dados . . . 8
5.2 Tabelas e Campos . . . 9

6 Funcionalidades do Sistema **9**
6.1 Cadastro de Músicas . . . 9
6.2 Consulta e Busca . . . 10
6.3 Edição de Dados . . . 10
6.4 Exclusão de Dados . . . 10

7 Fluxo de Funcionamento **10**
7.1 Inicialização do Sistema . . . 10
7.2 Fluxo Principal de Uso . . . 10
7.3 Interação entre Módulos . . . 11

8 Detalhamento Técnico **11**
8.1 Principais Arquivos . . . 11
8.2 Funções Relevantes . . . 12
8.3 Lógica de Funcionamento Interno . . . 13

9 Instalação e Execução **14**
9.1 Requisitos do Sistema . . . 14
9.2 Como Instalar . . . 14
9.3 Como Executar . . . 14

9.4 Geração de Executável 14

10 Tratamento de Erros e Validações 15
10.1 Comportamentos Esperados pelo Sistema 15
10.1.1 Busca sem Resultado 15
10.1.2 Arquivo Inexistente no Caminho Registrado 15
10.1.3 Campos Obrigatórios e Opcionais no Cadastro 15
10.1.4 Confirmação antes da Exclusão 15
10.1.5 Inicialização do Banco de Dados 15
10.2 Limitações no Tratamento de Erros 16

11 Limitações Conhecidas 16

12 Considerações de Segurança 16
12.1 Proteção contra Injeção de SQL 16
12.2 Armazenamento Local dos Dados 16
12.3 Ausência de Autenticação 17

13 Histórico de Versões 17

14 Melhorias Futuras 17

Contato e Repositório 18

1. Introdução

1.1 Objetivo do Documento

O presente documento tem como finalidade descrever de forma técnica e estruturada o sistema **Gralha**, uma aplicação desktop voltada para o gerenciamento de arquivos musicais. Este documento é destinado primariamente aos desenvolvedores responsáveis pela manutenção, evolução e integração do sistema, bem como a qualquer agente técnico que necessite compreender a arquitetura, o funcionamento interno e as decisões de projeto adotadas durante o desenvolvimento.

O documento serve como referência oficial para:

- Compreensão da estrutura modular e da arquitetura do sistema;
- Manutenção e correção de defeitos;
- Evolução e adição de novas funcionalidades;
- Integração com ferramentas externas ou futuras versões.

1.2 Escopo do Sistema

O sistema Gralha contempla as seguintes funcionalidades no escopo de sua versão atual:

- Cadastro de músicas com informações como nome, artista, álbum, cifra, tablatura, áudio e partitura;
- Consulta e busca de músicas armazenadas no banco de dados local;
- Edição e atualização de registros existentes;
- Exclusão permanente de registros do banco de dados;
- Visualização de cifras e tablaturas e abertura de arquivos de áudio e partitura associados às músicas.

Estão fora do escopo desta versão:

- Sincronização com serviços de nuvem ou repositórios remotos;
- Funcionalidade de reprodução de áudio integrada à interface;
- Suporte a múltiplos usuários ou controle de acesso.

1.3 Definições e Termos

SQLite Banco de dados relacional leve, armazenado em arquivo local, sem necessidade de servidor dedicado.

Tkinter Biblioteca padrão do Python para criação de interfaces gráficas desktop.

CustomTkinter Extensão da biblioteca Tkinter que oferece componentes visuais modernos e customizáveis.

Pillow Biblioteca Python para processamento, manipulação e exibição de imagens.

CRUD Acrônimo para as operações básicas de persistência: *Create* (criar), *Read* (ler), *Update* (atualizar) e *Delete* (excluir).

Cifra Representação de uma peça musical por meio de acordes em formato textual.

Tablatura Representação simplificada de uma peça musical em formato de texto ASCII, indicando as posições das notas em um instrumento de cordas.

Partitura Representação formal de uma composição musical em notação padrão, armazenada em formato PDF no sistema.

PyInstaller Ferramenta para empacotamento de aplicações Python em executáveis independentes.

.db Extensão de arquivo utilizada pelo SQLite para armazenar bancos de dados locais.

2. Visão Geral do Sistema

2.1 Descrição Geral

O sistema Gralha é uma aplicação desktop desenvolvida na linguagem Python, destinada ao gerenciamento centralizado de arquivos musicais. Por meio de uma interface gráfica intuitiva, o sistema permite ao usuário cadastrar, consultar, editar e excluir músicas, associando a cada registro informações como artista, álbum, cifra, tablatura, áudio e partitura.

A aplicação opera de forma completamente local, utilizando um banco de dados SQLite para persistência das informações, sem necessidade de conexão à internet ou instalação de serviços externos.

2.2 Problema que o Sistema Resolve

Músicos e estudantes de música frequentemente enfrentam dificuldades para organizar suas partituras, tablaturas e arquivos de áudio, especialmente quando tais recursos estão distribuídos em diferentes pastas, dispositivos ou em formato físico. A ausência de um ambiente centralizado dificulta o acesso rápido aos materiais e compromete a produtividade durante os estudos ou apresentações.

O sistema Gralha foi desenvolvido para solucionar essa problemática, centralizando todas as informações musicais em um único ambiente digital, organizado e de fácil acesso. Dessa forma, o usuário pode localizar rapidamente qualquer música cadastrada e acessar seus arquivos associados diretamente pela interface do sistema.

2.3 Público-Alvo

O sistema é destinado a músicos, estudantes de música e entusiastas que desejam organizar sua coleção musical de forma digital e estruturada. Não é exigido conhecimento técnico avançado para a utilização das funcionalidades disponíveis na interface gráfica. O sistema é projetado para ser intuitivo e acessível a usuários com diferentes níveis de familiaridade com ferramentas computacionais.

2.4 Funcionalidades Principais

O sistema Gralha disponibiliza as seguintes funcionalidades principais:

- Cadastro de novas músicas com informações detalhadas, incluindo nome, artista, álbum, ano, cifra e tablatura;
- Busca e filtragem de músicas por nome, artista ou álbum, com ordenação por relevância;
- Edição de informações de músicas já cadastradas;
- Exclusão de registros do banco de dados, com confirmação prévia;
- Associação de arquivos de áudio e partitura (PDF) a cada música;
- Visualização de cifras e tablaturas diretamente pela interface;
- Abertura de arquivos de áudio e partitura por meio do aplicativo padrão do sistema operacional.

3. Tecnologias Utilizadas

3.1 Linguagens de Programação

O sistema foi desenvolvido integralmente utilizando a linguagem de programação **Python**, na versão 3.10 ou superior. A escolha pela linguagem justifica-se por sua simplicidade sintática, ampla disponibilidade de bibliotecas para desenvolvimento de aplicações desktop e facilidade de manutenção do código-fonte.

3.2 Bibliotecas e Frameworks

As principais bibliotecas utilizadas no desenvolvimento do sistema estão descritas na Tabela 1.

Tabela 1: Bibliotecas utilizadas no sistema Gralha

Biblioteca	Descrição
Tkinter	Biblioteca padrão do Python para criação da interface gráfica desktop. Responsável pela estrutura das janelas, botões, entradas de texto e demais componentes visuais do sistema.
CustomTkinter	Extensão do Tkinter que fornece componentes visuais modernizados, aprimorando a estética da interface do usuário.
SQLite3	Módulo padrão do Python para comunicação com bancos de dados SQLite. Utilizado para executar operações de inserção, consulta, atualização e exclusão de registros.
Pillow	Biblioteca para processamento e manipulação de imagens no Python. Utilizada para exibição de recursos gráficos na interface, como logotipo e ícones.
OS	Módulo padrão do Python para operações com o sistema de arquivos, como verificação de existência de arquivos e abertura de documentos externos.

3.3 Banco de Dados

O sistema utiliza o **SQLite** como banco de dados. Essa tecnologia foi adotada por ser leve, de fácil integração com Python e por não exigir a instalação ou configuração de um servidor de banco de dados externo. Os dados são persistidos localmente em um arquivo com extensão `.db`, armazenado no diretório de dados do usuário do sistema operacional, no caminho `%APPDATA%\Gralha\musicas.db` (Windows).

3.4 Ferramentas de Desenvolvimento

As ferramentas utilizadas durante o processo de desenvolvimento do sistema estão listadas a seguir:

- **Visual Studio Code**: ambiente de desenvolvimento integrado utilizado para escrita e organização do código-fonte;
- **Git**: sistema de controle de versão utilizado para rastreamento de alterações no código;
- **GitHub**: plataforma de hospedagem do repositório remoto do projeto;
- **PyInstaller**: ferramenta utilizada para geração do executável da aplicação.

4. Arquitetura do Sistema

4.1 Visão Geral da Arquitetura

O sistema Gralha adota uma arquitetura modular estruturada em três camadas funcionais distintas:

- **Camada de Interface** (*View*): responsável pela apresentação visual e pela captura das interações do usuário. Engloba os arquivos do diretório `interface/`;
- **Camada de Serviços** (*Service*): responsável pelo processamento da lógica de negócio e pela mediação entre a interface e o banco de dados. Representada pelo módulo `music_service.py`;
- **Camada de Dados** (*Data*): responsável pela criação, conexão e operações no banco de dados. Representada pelo módulo `database.py` e pelo arquivo `musicas.db`.

A comunicação entre as camadas ocorre de forma unidirecional: a interface aciona a camada de serviços, que por sua vez executa operações na camada de dados. Os resultados percorrem o caminho inverso até serem exibidos ao usuário.

4.2 Estrutura de Diretórios

A estrutura de diretórios do projeto está organizada conforme apresentado abaixo:

```
gralha/
 assets/
    icons/               # Ícones utilizados nos botões da interface
    logo.png             # Logotipo do sistema
 database/
    database.py          # Conexão e criação do banco de dados
    musicas.db    # Gerado em %APPDATA%\Gralha\musicas.db
 interface/
    main_window.py       # Janela principal do sistema
    add_music_window.py  # Janela de cadastro de músicas
    edit_music_window.py # Janela de edição e exclusão de músicas
 services/
    music_service.py     # Lógica de negócio e acesso ao banco de dados
 main.py                  # Ponto de entrada da aplicação
```

4.3 Organização dos Módulos

A responsabilidade de cada módulo principal do sistema está descrita na Tabela 2.

Tabela 2: Descrição dos módulos do sistema

Módulo	Responsabilidade
`main.py`	Ponto de entrada da aplicação. Inicializa o banco de dados e carrega a interface principal.
`main_window.py`	Interface gráfica principal. Gerencia a busca de músicas e a navegação entre janelas.
`add_music_window.py`	Interface para o cadastro de novas músicas. Coleta dados e arquivos e os envia para a camada de serviços.
`edit_music_window.py`	Interface para edição e exclusão de músicas existentes.
`music_service.py`	Camada de serviços. Executa operações CRUD no banco de dados por meio de comandos SQL.
`database.py`	Gerencia a conexão com o banco de dados e a criação da tabela principal.
`musicas.db`	Arquivo de banco de dados SQLite contendo os registros das músicas cadastradas.

4.4 Diagrama da Arquitetura

A Figura 1 ilustra a arquitetura geral do sistema, representando a comunicação entre as camadas.

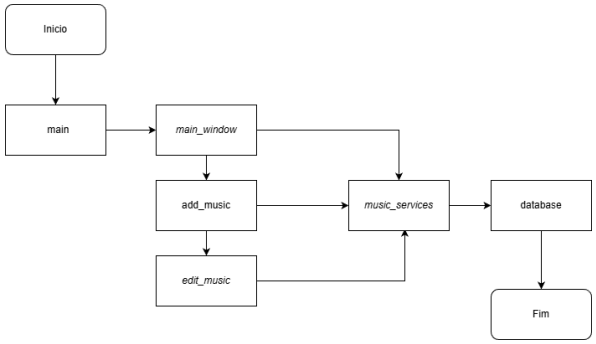

Figura 1: Diagrama de arquitetura do sistema Gralha

5. Modelagem de Dados

5.1 Estrutura do Banco de Dados

O sistema utiliza um banco de dados relacional baseado em SQLite. Os dados são organizados em tabelas que representam as entidades do domínio da aplicação. O banco de dados é armazenado localmente no diretório de dados do usuário do sistema operacional, seguindo o padrão de aplicações desktop para Windows. O caminho completo do arquivo é:

`%APPDATA%\Gralha\musicas.db`

Esse diretório é criado automaticamente pela aplicação na primeira execução, caso ainda não exista. O arquivo é acessado diretamente pela aplicação sem intermediação de servidor.

5.2 Tabelas e Campos

O banco de dados é composto por uma tabela principal denominada `musicas`, cuja estrutura está descrita na Tabela 3.

Tabela 3: Estrutura da tabela `musicas`

Campo	Tipo	Descrição
`id`	INTEGER	Identificador único do registro, gerado automaticamente (chave primária).
`nome`	TEXT	Nome ou título da música cadastrada.
`artista`	TEXT	Nome do artista ou banda responsável pela composição.
`album`	TEXT	Nome do álbum ao qual a música pertence.
`ano`	INTEGER	Ano de lançamento da música.
`cifra`	TEXT	Conteúdo textual da cifra da música, quando disponível.
`tablatura`	TEXT	Conteúdo textual da tablatura da música em formato ASCII, quando disponível.
`caminho_audio`	TEXT	Caminho absoluto para o arquivo de áudio associado à música.
`caminho_partitura`	TEXT	Caminho absoluto para o arquivo PDF de partitura associado à música.

6. Funcionalidades do Sistema

6.1 Cadastro de Músicas

O sistema permite o cadastro de novas músicas por meio de uma janela dedicada, acessada a partir da interface principal. O usuário preenche os campos referentes ao nome, artista, álbum, ano, cifra e tablatura, e pode opcionalmente associar arquivos de áudio e partitura ao registro.

Após o preenchimento, os dados são enviados para a camada de serviços, que valida as informações e executa a inserção no banco de dados. Uma vez cadastrada, a música torna-se disponível para consulta, edição e exclusão.

6.2 Consulta e Busca

O sistema disponibiliza uma funcionalidade de busca na interface principal, permitindo ao usuário localizar músicas cadastradas com base no nome, artista ou álbum. A busca utiliza o operador `LIKE` do SQL com critério composto entre os três campos, retornando todos os registros que contenham o termo informado. Os resultados são ordenados por relevância, com prioridade para registros cujo nome comece com o texto digitado, seguidos dos demais resultados em ordem alfabética.

Quando a busca retorna um único resultado, a interface exibe diretamente o card de detalhes da música. Quando retorna múltiplos resultados, exibe a lista interativa para seleção. Os detalhes incluem artista, álbum, ano e atalhos para abertura dos arquivos e visualização de cifra e tablatura.

6.3 Edição de Dados

O sistema permite a edição de registros já cadastrados. O usuário pode selecionar uma música, acessar a janela de edição e modificar os campos desejados. Após a confirmação, as alterações são processadas pela camada de serviços e persistidas no banco de dados por meio de um comando de atualização SQL.

6.4 Exclusão de Dados

O sistema possibilita a exclusão permanente de músicas armazenadas. Ao selecionar um registro e acionar a opção de exclusão, o sistema exibe uma caixa de diálogo de confirmação antes de prosseguir. Após a confirmação do usuário, a solicitação é encaminhada para a camada de serviços, que executa a remoção do registro no banco de dados. A operação é irreversível e o registro não pode ser recuperado após a exclusão.

7. Fluxo de Funcionamento

7.1 Inicialização do Sistema

Ao executar a aplicação, o arquivo `main.py` é carregado como ponto de entrada. Durante a inicialização, o sistema verifica a existência do banco de dados e, caso necessário, executa a criação da tabela principal por meio da função `criar_tabela()` presente no módulo `database.py`.

Em seguida, a interface gráfica principal é instanciada e exibida ao usuário por meio da função `iniciar_interface()`, localizada em `main_window.py`, dando início ao ciclo de interação.

7.2 Fluxo Principal de Uso

A partir da interface principal, o usuário pode realizar as seguintes operações:

- **Busca**: realizada diretamente na janela principal (`main_window.py`), por meio do campo de entrada e do botão de busca;
- **Cadastro**: direciona o usuário para a janela `add_music_window.py`, onde os dados e arquivos da nova música são inseridos;

- **Edição e Exclusão**: direciona o usuário para a janela `edit_music_window.py`, que permite a modificação ou remoção de um registro existente.

Após a execução de qualquer operação, o sistema processa a requisição por meio da camada de serviços e atualiza a interface para refletir o estado atual do banco de dados.

7.3 Interação entre Módulos

A comunicação entre os módulos do sistema ocorre de forma estruturada em camadas. A interface gráfica captura as ações do usuário e as encaminha para o módulo de serviços (`music_service.py`), que é responsável pelo processamento da lógica de negócio.

O módulo de serviços executa as operações pertinentes no banco de dados (`musicas.db`), realizando inserções, consultas, atualizações ou exclusões conforme a operação solicitada. Os dados resultantes retornam à camada de interface, que os exibe ao usuário de forma organizada.

Esse modelo garante a separação entre interface, lógica e persistência de dados, facilitando a manutenção e a evolução do sistema.

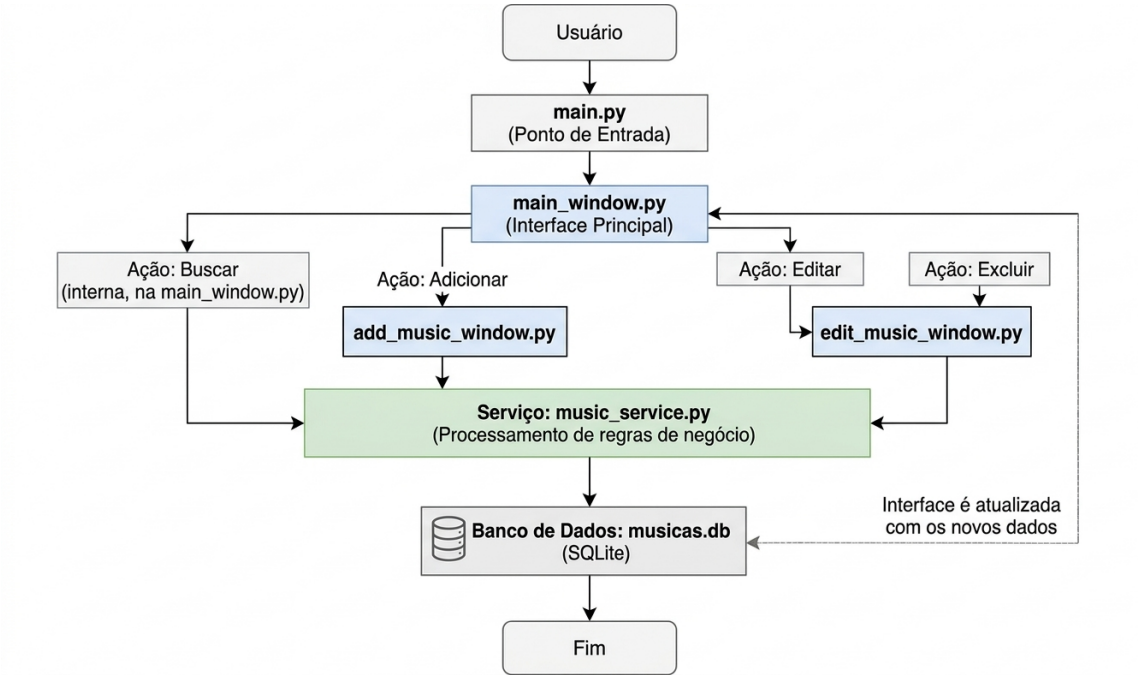

Figura 2: Fluxograma de funcionamento do sistema Gralha

O fluxo de funcionamento do sistema está representado no diagrama apresentado anteriormente, que ilustra de forma visual a interação entre os componentes.

8. Detalhamento Técnico

8.1 Principais Arquivos

O sistema é composto pelos seguintes arquivos principais, organizados por responsabilidade:

- `main.py`: responsável pela inicialização do sistema, execução da criação do banco de dados e chamada da interface principal;
- `main_window.py`: implementa a interface gráfica principal e gerencia a interação do usuário com as funcionalidades de busca e navegação;
- `add_music_window.py`: responsável pela interface de cadastro de novas músicas, incluindo a seleção de arquivos externos;
- `edit_music_window.py`: responsável pela interface de edição e exclusão de músicas existentes;
- `music_service.py`: contém a lógica de negócio do sistema, intermediando a comunicação entre a interface gráfica e o banco de dados;
- `database.py`: gerencia a conexão com o banco de dados e a criação da estrutura de tabelas;
- `musicas.db`: arquivo de banco de dados SQLite gerado automaticamente em `%APPDATA%\Gralha\musicas.db` na primeira execução da aplicação.

8.2 Funções Relevantes

As principais funções do sistema são descritas na Tabela 4.

Tabela 4: Principais funções do sistema Gralha

Função	Módulo	Descrição
`criar_tabela()`	database.py	Cria a tabela `musicas` no banco de dados, caso ainda não exista.
`conectar_banco()`	database.py	Estabelece e retorna a conexão com o banco de dados SQLite.
`listar_musicas()`	music_service.py	Retorna todas as músicas cadastradas, ordenadas por nome.
`filtrar_musicas()`	music_service.py	Filtra músicas por nome, artista ou álbum, com ordenação por relevância. Utilizada pela busca na interface principal.
`add_musica()`	music_service.py	Insere um novo registro de música no banco de dados.
`editar_musica()`	music_service.py	Atualiza os dados de um registro existente no banco de dados.
`excluir_musica()`	music_service.py	Remove permanentemente um registro do banco de dados.
`iniciar_interface()`	main_window.py	Inicializa e exibe a janela principal da aplicação.
`abrir_janela_adicionar()`	add_music_window.py	Exibe a janela de cadastro de nova música.
`abrir_janela_editar()`	edit_music_window.py	Exibe a janela de edição de uma música existente.

8.3 Lógica de Funcionamento Interno

O funcionamento interno do sistema segue um fluxo estruturado e coerente com a arquitetura em camadas adotada. As ações realizadas pelo usuário na interface gráfica acionam funções específicas, que encaminham as requisições para o módulo de serviços (`music_service.py`).

Esse módulo é responsável por validar e processar os dados recebidos, executando operações no banco de dados por meio de comandos SQL parametrizados. A parametrização das consultas garante a segurança contra injeções SQL e a integridade dos dados armazenados.

Após a execução das operações, os resultados são retornados à camada de interface, que atualiza os elementos visuais correspondentes. Esse modelo assegura a separação entre interface e lógica de negócio, tornando o sistema organizado, de fácil manutenção e extensível para futuras evoluções.

9. Instalação e Execução

9.1 Requisitos do Sistema

Para executar o sistema Gralha, são necessários os seguintes requisitos mínimos:

- Sistema operacional: Windows 10 ou superior;
- Python 3.10 ou superior instalado no ambiente;
- As bibliotecas Tkinter e SQLite3 já estão incluídas na instalação padrão do Python;
- Bibliotecas adicionais: `customtkinter` e `Pillow`.

Não é necessária a instalação de servidor de banco de dados externo, pois o sistema utiliza SQLite local.

9.2 Como Instalar

Para instalar o sistema, siga os passos abaixo:

1. Faça o download ou clone o repositório do projeto:

```
git clone https://github.com/seu-usuario/gralha.git
```

2. Acesse o diretório do projeto:

```
cd gralha
```

3. (Opcional) Crie e ative um ambiente virtual:

```
python -m venv venv
venv\Scripts\activate   # Windows
source venv/bin/activate  # Linux / macOS
```

4. Instale as dependências externas necessárias:

```
pip install customtkinter pillow
```

9.3 Como Executar

Para iniciar o sistema, execute o seguinte comando no terminal a partir do diretório raiz do projeto:

```
python main.py
```

Após a execução do comando, a interface gráfica será carregada automaticamente.

9.4 Geração de Executável

O sistema pode ser convertido em um executável independente para facilitar sua distribuição em ambientes Windows, utilizando a ferramenta **PyInstaller**. Para gerar o executável, execute o seguinte comando no terminal:

```
pyinstaller --onefile --windowed --icon=assets/logo.ico --add-
   data "assets;assets" --name=Gralha main.py
```

Após a conclusão do processo, o arquivo executável `main.exe` será gerado no diretório `dist/`. Este arquivo pode ser distribuído e executado em sistemas Windows sem a necessidade de instalação prévia do Python ou de suas dependências.

10. Tratamento de Erros e Validações

10.1 Comportamentos Esperados pelo Sistema

O sistema implementa verificações em pontos críticos do fluxo de execução, garantindo que situações de erro sejam tratadas de forma controlada. Os principais comportamentos estão descritos a seguir.

10.1.1 Busca sem Resultado

Quando a busca por uma música não retorna registros correspondentes no banco de dados, a interface exibe a mensagem *"Música não encontrada"* ao usuário. Nenhuma exceção é lançada e o sistema permanece em estado operacional normal, aguardando nova interação.

10.1.2 Arquivo Inexistente no Caminho Registrado

Antes de abrir um arquivo de áudio ou partitura, o sistema verifica a existência do arquivo no caminho armazenado, por meio da função `os.path.exists()`. Caso o arquivo não seja localizado, uma mensagem de aviso é exibida ao usuário informando o caminho não encontrado.

10.1.3 Campos Obrigatórios e Opcionais no Cadastro

Na janela de cadastro (`add_music_window.py`), os campos `nome` e `artista` são obrigatórios; os demais (álbum, ano, cifra, tablatura, áudio e partitura) são opcionais. Caso campos opcionais não sejam preenchidos ou arquivos não sejam selecionados, os valores correspondentes são registrados como vazios ou nulos no banco de dados. O campo `ano` possui validação de formato, exigindo um valor numérico de quatro dígitos.

10.1.4 Confirmação antes da Exclusão

Antes de executar a exclusão de um registro, o sistema exibe uma caixa de diálogo de confirmação (`messagebox.askyesno`), solicitando a aprovação explícita do usuário. A remoção só é efetivada após a confirmação positiva.

10.1.5 Inicialização do Banco de Dados

Durante a inicialização da aplicação, a função `criar_tabela()` utiliza a instrução `CREATE TABLE IF NOT EXISTS`, garantindo que a tabela principal seja criada apenas se ainda não existir. Isso evita erros de duplicidade em execuções subsequentes da aplicação.

10.2 Limitações no Tratamento de Erros

Na versão atual, o sistema não implementa tratamento de exceções explícito (`try/except`) nas operações de banco de dados. Falhas decorrentes de permissões de acesso ao arquivo `.db`, corrupção do banco ou caminho incorreto podem resultar em encerramento inesperado da aplicação. Recomenda-se a implementação de blocos de tratamento de exceção em versões futuras.

11. Limitações Conhecidas

O sistema Gralha, em sua versão atual, apresenta as seguintes limitações identificadas durante o desenvolvimento:

- **Variáveis globais na janela de cadastro**: o módulo `add_music_window.py` utiliza variáveis globais (`caminho_audio`, `caminho_partitura`) para armazenar os caminhos dos arquivos selecionados. Essa abordagem pode causar comportamento inesperado caso a janela seja aberta múltiplas vezes na mesma sessão, pois os valores anteriores persistem entre as chamadas;
- **Dependência do caminho absoluto dos arquivos**: os caminhos de áudio e partitura são armazenados como fornecidos pelo sistema operacional no momento da seleção. Caso os arquivos sejam movidos ou o sistema seja transferido para outro dispositivo, os vínculos serão perdidos;
- **Compatibilidade do comando `os.startfile()`**: a função utilizada para abrir arquivos de áudio e partitura é exclusiva do sistema operacional Windows, tornando essa funcionalidade inoperante em ambientes Linux e macOS sem adaptação.

12. Considerações de Segurança

12.1 Proteção contra Injeção de SQL

Todas as operações de consulta e inserção no banco de dados utilizadas pelo módulo `music_service.py` fazem uso de parâmetros substituídos por marcadores (`?`), conforme o padrão da biblioteca `sqlite3` do Python. Essa abordagem garante que os valores fornecidos pelo usuário sejam tratados como dados e não como parte da instrução SQL, protegendo o sistema contra ataques de injeção de SQL.

12.2 Armazenamento Local dos Dados

Os dados do sistema são armazenados exclusivamente em um arquivo local (`musicas.db`), sem transmissão para servidores externos ou serviços de nuvem. Isso reduz a superfície de exposição a ameaças externas, porém implica que a segurança dos dados depende integralmente das permissões e do controle de acesso do sistema operacional do dispositivo onde a aplicação está instalada.

12.3 Ausência de Autenticação

O sistema não implementa mecanismos de autenticação ou controle de acesso. Qualquer usuário com acesso ao dispositivo pode executar a aplicação e realizar operações de leitura, escrita e exclusão nos dados armazenados. Para ambientes compartilhados, recomenda-se avaliar a implementação de controle de acesso em versões futuras.

13. Histórico de Versões

Tabela 5: Histórico de versões do sistema Gralha

Versão	Data	Responsável	Descrição
1.0	22 de março de 2026	—	Versão inicial do sistema. Implementação das funcionalidades de cadastro, busca, edição, exclusão e abertura de arquivos musicais.

14. Melhorias Futuras

Com base nas limitações identificadas na versão atual e nas necessidades observadas durante o desenvolvimento, as seguintes melhorias são sugeridas para versões futuras do sistema:

- **Refatoração das variáveis globais**: substituição das variáveis globais em `add_music_window.py` por uma abordagem orientada a objetos, encapsulando o estado da janela em atributos de classe;
- **Armazenamento relativo de caminhos**: implementação de cópia dos arquivos selecionados para um diretório interno do projeto, armazenando caminhos relativos no banco de dados para garantir portabilidade entre dispositivos;
- **Tratamento de exceções**: adição de blocos `try/except` nas operações de banco de dados e manipulação de arquivos, com exibição de mensagens de erro informativas ao usuário;
- **Compatibilidade multiplataforma**: substituição de `os.startfile()` por uma solução compatível com Windows, Linux e macOS, como o módulo `subprocess` com detecção do sistema operacional;
- **Reprodução de áudio integrada**: implementação de um player de áudio interno à interface, eliminando a dependência de aplicativos externos para reprodução das faixas associadas.

Contato e Repositório

Para acesso ao repositório do projeto, download da versão mais recente ou contato com o desenvolvedor responsável, utilize os canais abaixo:

GitHub: github.com/adrielsoares2309/gralha

E-mail: adrielsoares2309@gmail.com

Documentação Técnica — Sistema Gralha — Versão 1.0 — 22 de março de 2026